Ticking Backwards — CTF Write-up

Category: Digital Forensics / Windows Artifact Analysis (NTFS) **Points:** 250 **Difficulty:** Medium **Author:** k4p3re **Flag:** `r00t{t1m3l1n3_tru7h_1n_th3_mft}`

Challenge Brief

> Reconstruct the timeline, and recover the password on the archive the data was packaged into.
>
> You're handed three raw NTFS artifacts pulled from a compromised host:
>
> - `$MFT` — the Master File Table
> - `$J` — the USN change journal
> - `WMIPRVSE.EXE-A1B2C3D4.pf` — a Windows Prefetch file

No disk image, no memory dump — just three raw filesystem metadata files. The whole case has to be rebuilt from timestamps, journal records, and whatever's sitting resident inside the MFT itself.

Step 1 — Unpack and get oriented

```
┌──(enos㉿kali)-[~/ticking_backwards]
└─$ unzip -l Ticking_backwards.zip
Archive:  Ticking_backwards.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-10-28 23:06   Ticking backwards/
     1184  2026-10-28 23:06   Ticking backwards/$J
    44032  2026-10-28 23:06   Ticking backwards/$MFT
      536  2026-10-28 23:06   Ticking backwards/WMIPRVSE.EXE-A1B2C3D4.pf
---------                     -------
    45752                     4 files

┌──(enos㉿kali)-[~/ticking_backwards]
└─$ unzip -o Ticking_backwards.zip -d work/
  inflating: work/Ticking backwards/$J
  inflating: work/Ticking backwards/$MFT
  inflating: work/Ticking backwards/WMIPRVSE.EXE-A1B2C3D4.pf
```

No `mmls`, `analyzeMFT`, `MFTECmd`, or `construct` module on this box, and no network to fetch them — so every parser here is hand-rolled Python against the raw NTFS record formats.

Step 2 — Identify the artifacts by their magic bytes

```
┌──(enos㉿kali)-[~/ticking_backwards]
└─$ xxd WMIPRVSE.EXE-A1B2C3D4.pf | head -1
bash: xxd: command not found

┌──(enos㉿kali)-[~/ticking_backwards]
└─$ python3 -c "print(open('WMIPRVSE.EXE-A1B2C3D4.pf','rb').read(8).hex())"
1700000053434341

┌──(enos㉿kali)-[~/ticking_backwards]
└─$ python3 -c "print(open('\$MFT','rb').read(4))"
b'FILE'

┌──(enos㉿kali)-[~/ticking_backwards]
└─$ python3 -c "print(open('\$J','rb').read(16).hex())"
00000000000000000000000000000000
```

- `17 00 00 00 'SCCA'` → Prefetch **version 23** (Windows 7/Server 2008R2), **uncompressed** — can parse directly, no MAM decompression needed.
- `FILE` signature → standard 1024-byte MFT records, 43 records total (44032 ÷ 1024).
- `$J` starts as a run of zeros — normal; USN journal is a sparse file, real records start further in.

Step 3 — Parse the $MFT (with proper fixup handling)

First pass without applying the NTFS **Update Sequence Array (USA)** fixup produced garbage at every 512-byte sector boundary inside larger records — classic beginner mistake. Rewrote the parser to restore the original sector-end bytes from the USA before reading attributes:

```
┌──(enos㉿kali)-[~/ticking_backwards]
└─$ python3 parse_mft.py '$MFT' | head -20
=== Record 30 seq=1 flags=1 (in-use=True, dir=False) ===
  SI: created=2026-09-10 02:11:05.xxxxx modified=2026-09-10 02:11:05...
  FN: name='~DF8841.tmp' parent=5
  DATA resident (64 bytes): b'token=H4sIAAAAAAAC/8tNzMwrSc1LzEtO1S3Jz07N...'

=== Record 35 seq=1 flags=1 (in-use=True, dir=False) ===
  FN: name='svc_cache.dat' parent=5
  DATA resident (52 bytes): b'key=H4sIAAAAAAAC/0tKTM4uLdDNTq3UNTCwtDQEAMy...'

=== Record 41 seq=1 flags=1 (in-use=True, dir=False) ===
  FN: name='collect.bat' parent=5
  DATA resident (808 bytes): b'@echo off\r\nREM nightly finance
sync\r\n...\r\nset
B=H4sIAAAAAAAC/ysyMCipLjHMNc4xzDOOLykqNc+IN8yLL8kwjs9NK6kFALg4Hn4fAAAA\r\npow
ershell ... -p%P% ...'
```

Three resident `$DATA` attributes sitting directly inside the MFT records: a staged `token=`, a staged `key=`, and a `collect.bat` script that builds a 7-Zip password at runtime from a gzip+Base64 blob.

Step 4 — Decode the staged secrets

```
┌──(enos㉿kali)-[~/ticking_backwards]
└─$ python3 -c "
import base64, gzip
blobs = {
  'token': 'H4sIAAAAAAAC/8tNzMwrSc1LzEtO1S3Jz07N0zU3NzYEAGpkNqgWAAAA',
  'key':   'H4sIAAAAAAAC/0tKTM4uLdDNTq3UNTCwtDQEAMy/ZOkQAAAA',
  'B':
'H4sIAAAAAAAC/ysyMCipLjHMNc4xzDOOLykqNc+IN8yLL8kwjs9NK6kFALg4Hn4fAAAA',
}
for name, b64 in blobs.items():
    print(name, '=>', gzip.decompress(base64.b64decode(b64)))
"
token => b'maintenance-token-7731'
key   => b'backup-key-00991'
B     => b'r00t{t1m3l1n3_tru7h_1n_th3_mft}'
```

⚠️ **Two decoys.** `token` and `key` are staged, plausible-looking credentials dropped and deleted days apart — noise to chase. Only `B`, the variable `collect.bat` actually feeds to `7z -p%P%` when building the exfil archive, decodes to the real password/flag.

Step 5 — Reconstruct the real timeline from the $J USN journal

`$SI` (Standard Information) timestamps are trivially forged by anyone with `SetFileTime`-level access — `wmiprvse.exe`'s `$SI` creation date was stomped back to 2009 to blend in with genuine OS files. The USN journal records actual filesystem events as they happened and is far harder to tamper with:

```
┌──(enos㉿kali)-[~/ticking_backwards]
└─$ python3 parse_usnjournal.py '$J'
2026-09-10 02:11:05  ~DF8841.tmp         FILE_CREATE
2026-09-10 02:11:09  ~DF8841.tmp         FILE_DELETE
2026-09-14 08:55:12  svc_cache.dat       FILE_CREATE
2026-09-14 08:56:01  svc_cache.dat       FILE_DELETE
2026-09-14 09:42:17  wmiprvse.exe        FILE_CREATE
2026-09-14 09:44:02  collect.bat         FILE_CREATE
2026-09-14 09:44:30  collect.bat         FILE_DELETE
2026-09-14 09:45:10  exfil.7z            FILE_CREATE
```

That's the real chronology — drop decoy → drop decoy → drop the renamed 7-Zip binary → drop and run the exfil script → delete it → archive created, all inside about 3 minutes on 2026-09-14.

Step 6 — Corroborate execution with the Prefetch file

```
┌──(enos㉿kali)-[~/ticking_backwards]
└─$ python3 parse_prefetch.py WMIPRVSE.EXE-A1B2C3D4.pf
version        : 23 (Windows 7)
executable     : WMIPRVSE.EXE
file size      : 536 bytes
last run time  : 2026-09-14 09:47:33 UTC
```

A third, independent witness: the renamed `7z.exe` (impersonating the legit WMI provider host process) actually ran around 09:47 — right after the archive was built, consistent with the journal.

Step 7 — Confirm the flag

```
┌──(enos㉿kali)-[~/ticking_backwards]
└─$ echo "r00t{t1m3l1n3_tru7h_1n_th3_mft}"
r00t{t1m3l1n3_tru7h_1n_th3_mft}
```

Timeline Summary

Time (2026-09-xx)	Event
09-10 02:11:05–02:11:09	`~DF8841.tmp` staged & deleted — decoy `token=`
09-14 08:55:12–08:56:01	`svc_cache.dat` staged & deleted — decoy `key=`
09-14 09:42:17	`wmiprvse.exe` (renamed 7-Zip) dropped into `Windows\Temp`
09-14 09:44:02–09:44:30	`collect.bat` created, run, deleted — builds the real password
09-14 09:45:10	`exfil.7z` created, password-protected
09-14 09:47:33	Prefetch last-run confirms execution of the renamed binary

---

Key Takeaways

1. **Never trust `$STANDARD_INFORMATION` alone.** It's the easiest NTFS timestamp set to forge (any tool with `SetFileTime` access can stomp it). Here it was backdated to 2009 specifically to blend a dropped binary in with genuine OS files.
2. **`$FILE_NAME` timestamps + the `$J` USN journal are closer to ground truth.** Rebuilding the real sequence of events meant cross-referencing both against the forged `$SI` values — the mismatch itself was the tell.
3. **Resident `$MFT` attributes can hide real payload data.** Small files get their `$DATA` attribute stored directly inside the 1024-byte MFT record instead of allocated clusters — worth extracting and decoding *every* resident attribute, not just filenames and timestamps.

4. **Always apply the Update Sequence Array (USA) fixup before parsing MFT records.** Skipping it corrupts data at every 512-byte sector boundary and silently breaks any record larger than one sector.
5. **Prefetch is a third independent execution witness.** Even with the `$MFT` and `$J` timestomped or deleted, a `.pf` file's last-run time is a separate artifact that's awkward for an attacker to also scrub — useful for corroboration.
6. **Expect layered decoys in staged secrets.** Two of the three gzip+Base64 blobs recovered were plausible-looking bait (`maintenance-token-7731`, `backup-key-00991`); only the one embedded in the actual exfiltration script was the real password/flag.

.